OSS 2강 · 라이선스

Open Source Software 라이선스

자유에는 조건이 있다 — 저작권 위에서 지켜야 할 규칙들

목차

1 **라이선스란 무엇인가**
저작권·특허와 어떻게 다른가

2 **현황과 새로운 리스크**
오늘날 무엇이 쓰이고, AI는 무엇을 바꿨나

3 **준수사항과 의무**
반드시 지켜야 할 것들

4 **대표 라이선스 살펴보기**
GPL, LGPL, MPL, BSD, Apache

5 **실무에 적용하기**
비교, 양립성, 이중 라이선스, 관리

라이선스란 무엇인가

저작권·특허와 헷갈리기 쉽지만, 셋은 서로 다른 권리입니다

저작권 (Copyright)

창작과 동시에 자동으로 발생하는 권리.
저작물을 보호하고, 타인이 쓰려면 저작자의 허락이 필요.

특허 (Patent)

발명에 대해 별도로 출원해 인정받는 독점권.
제품·기술에 대한 배타적 권리를 부여.

라이선스 (License)

저작권자가 사용자에게 내리는 '사용 허가'.
정해진 조건 아래 사용·수정·배포의 자유 허용.



라이선스가 명시되지 않은 프로젝트를 사용하는 것은 위험합니다 — 법적 문제가 발생할 수 있습니다.

오늘날, 어떤 라이선스가 쓰이나

전 세계 약 2,000여 종의 OSS 라이선스가 있지만, 실무는 소수가 지배합니다

78%

전체 오픈소스 컴포넌트 중
Permissive 라이선스 비중 (2024년)

92%

MIT 라이선스 코드를 포함한
상용 코드베이스의 비율

22%

Copyleft(GPL 계열) 비중
— 과거 40%대에서 지속 하락

경향

MIT가 가장 널리 쓰이는 라이선스 1위 자리를 유지하고, **Apache 2.0**은 Kubernetes 등 클라우드 네이티브 진영에서 강세입니다.
GPL 계열은 Linux 커널 등 시스템 소프트웨어에 집중되며 비중이 꾸준히 줄고 있습니다.

출처: Black Duck OSSRA 2025·2026, Open Source Initiative(OSI) 2025 라이선스 랭킹

2 · 현황과 새 리스크

AI 코딩 도구가 불러온 새로운 리스크

라이선스 세탁(License Laundering)

68%

2026년 상용 코드베이스의 라이선스 충돌 발견율

54%

AI 생성 코드의 IP·라이선스 Risk를 검토하는 기업 비율 — 나머지 절반 가까이는 검토 없이 사용

2,675건

한 코드베이스에서 발견된 라이선스 충돌 건수 (최대 사례)

무엇이 문제인가

GitHub Copilot, Cursor 같은 AI 코딩 도구는 GPL 등 카피레프트 코드를 포함한 방대한 공개 저장소로 학습됩니다. 코드를 생성할 때 그 코드가 어떤 라이선스에서 유래했는지 정보가 남지 않기 때문에, 개발자는 자신도 모르게 카피레프트 코드를 라이선스 표시 없이 프로젝트에 끌어오게 됩니다.

출처: Black Duck 2026 OSSRA(Open Source Security and Risk Analysis) 리포트 · *1강 "검증자로서의 개발자" 메시지와 연결됩니다 — AI가 짜준 코드일수록 출처 확인이 중요합니다.*

공통 준수사항

OSS라면 라이선스 종류와 무관하게 지켜야 하는 세 가지

1

저작권 문구 유지

소스에 담긴 저작권 문구를 그대로 유지해야 합니다. 소스를 수정했다면, 기존 라이선스 아래에 수정 사실·일자·수정자의 저작권 문구를 추가로 명시합니다.

2

상표권 보호

OSS의 제품명은 상표권(trademark)으로 보호됩니다. 동일한 이름을 자신의 제품·서비스명으로 사용하면 상표권 침해가 될 수 있습니다.

3

라이선스 양립성 확인

서로 다른 라이선스의 코드를 결합할 때, 두 라이선스의 요구사항이 상충하지 않는지 확인해야 합니다. 상충하면 결합된 프로그램을 배포하는 것 자체가 불가능해질 수 있습니다.

라이선스에 따라 달라지는 의무사항

같은 셋을 요구하는 라이선스도, 일부만 요구하는 라이선스도 있습니다

사용 여부 명시

"본 SW는 오픈소스 OO를 사용하였습니다"를 매뉴얼 등에 고지.

GPL·LGPL·MPL — 소스를 어디서 구할 수 있는지까지 구체적으로 고지

MIT·BSD·Apache — 코드에 저작권 문구 유지로 충분한 경우가 많음

소스코드 공개

수정·추가한 부분의 소스코드를 공개해야 하는 경우가 있습니다.

대표 라이선스: GPL

공개 범위는 라이선스마다 상이

특허 조항

Patent Grant — 저작권자의 특허권도 함께 무상 제공.

Patent Retaliation —사용자가 특허소송을 걸면 라이선스가 자동 종료되고 로열티가 부과되는 등 보복 조항이 적용됩니다.

라이선스 스펙트럼

Permissive에서 Strong Copyleft까지, 자유의 정도가 다릅니다

Permissive (자유로움) — **Restrictive (제약이 강함)**



Give Me Credit

Permissive

BSD, MIT, Apache License

수정해도 소스공개 의무 없음.
저작권 표시만 유지하면 자유롭게 사용

Give Me Fixes

Weak Copyleft

LGPL, MPL, EPL

수정한 부분만 공개 의무.
연결·결합된 나머지 코드는 자유롭게 배포 가능

Give Me Everything

Strong Copyleft

GPL, Affero GPL

결합된 전체 프로그램의 소스코드를 동일 라이선스로 공개해야 함

Copyright는 저작물을 보호하고 사용자가 저작자의 허락을 받도록 하는 개념이고,
Copyleft는 자유로운 사용·배포를 허용하되 변경된 소스코드도 동일 라이선스로 공개하도록 함으로써 파생물이 계속 자유롭게 유지되도록 합니다.

GPL — 가장 강력한 Copyleft 라이선스

Free Software Foundation(FSF)이 만든, 가장 제약이 강한 라이선스

GPL v2.0

- SW를 수정하거나 라이브러리를 링크(정적·동적 모두)하면, 개발된 SW 전체를 공개하고 GPL로 배포해야 함
- 내부적으로만 사용(배포하지 않음)하는 경우에는 공개 의무 없음
- 기업 입장에서는 영업 비밀(성능 개선, 핵심 기능)까지 공개해야 하므로 사업 전략과 상충될 수 있음

대표 사례: Linux Kernel, MariaDB, WordPress

GPL v3.0 — v2 대비 달라진 점

- Tivoization 금지 조항 추가 – 소스코드만 주는 걸로 끝나지 않고, 사용자가 그 코드를 고쳐서 만든 프로그램을 실제 기기에 설치해 실행할 수 있게 해줘야 함 (제조사가 몰래 잠금장치를 걸어 수정판 실행을 막는 걸 금지)
- 특허 조항 추가 — Patent Grant, Patent Retaliation
- 호환성 조항 추가 — Apache License 2.0과도 양립 가능해짐

Linux 커널은 지금도 GPLv2만 사용합니다 — 토발즈가 Tivoization 금지 조항에 반대해 v3로 전환하지 않았습니다.

LGPL · MPL — 절충형 Copyleft

GPL만큼 강하지 않게, 기업이 받아들이기 쉽게 만든 라이선스

LGPL 2.1

GNU Lesser General Public License

- FSF가 공개SW 확산을 장려하기 위해 전략적으로 만든 라이선스
- 동적 링크(Dynamic Linking) — 실행 시점에 LGPL 라이브러리를 불러오면, 응용프로그램 자체의 소스코드는 공개 의무가 없음
- 정적 링크(Static Linking) — 라이브러리를 프로그램에 컴파일해 넣는 경우, 사용자가 다른 버전으로 재링크할 수 있도록 오브젝트 파일 등을 제공해야 함
- 그래서 기업 입장에서는 GPL보다 LGPL을 선호하는 경우가 많음

MPL

Mozilla Public License

- Netscape 브라우저의 소스코드를 공개하기 위해 개발된 라이선스
- MPL 코드를 수정하면, 수정한 코드(파일 단위)만 공개하고 MPL로 배포
- 다른 라이선스의 코드와 결합 가능 — 결합된 부분은 MPL이 아닌 다른 라이선스로 배포 허용
- Patent Grant · Patent Retaliation 특허 조항 포함

BSD · Apache — 대표적인 Permissive 라이선스

소스 공개 의무가 없어, 상용 소프트웨어에서 널리 쓰입니다

BSD License

Berkeley Software Distribution License

- 소스코드 공개 의무가 없는 대표적인 라이선스(미국 정부 재원으로 개발)
- 누구든 BSD 기반 코드로 새 프로그램을 개발한 뒤, 소스코드를 공개하지 않고 판매할 수 있음
- 상용 소프트웨어에서 제약 없이 사용 가능 — MIT 라이선스와 유사한 권리

Apache License 2.0

Apache Software Foundation(ASF)

- 아파치 웹 서버를 포함한 ASF의 모든 SW에 적용되는 라이선스. 'Apache' 명칭에 대한 상표권을 침해해서는 안 됨
- Patent Grant — 기여자가 자신이 기여한 부분에 대해 가진 특허권을 사용자에게 무상으로 실시 허락
- Patent Retaliation — 사용자가 이 코드로 특허 소송을 걸면, 그 순간 부여받은 특허 라이선스가 자동 종료
- GPL 2.0과는 비양립, GPL 3.0과는 양립 가능

라이선스 한눈에 비교하기

사용자 관점에서 무엇이 허용되고, 무엇이 요구되는가?

구분	무료이용	소스코드 취득	소스코드 수정	배포 허용	2차 저작물 공개	독점SW와 결합	특허 조항
GNU GPL	O	O	O	O	O	X	△
GNU LGPL	O	O	O	O	O	O	△
MPL	O	O	O	O	O	O	O
BSD License	O	O	O	O	X	O	X
Apache License	O	O	O	O	X	O	O

△ 특허 조항은 버전에 따라 다릅니다 — GPLv2·LGPLv2.1은 명시적 특허 조항이 없고, GPLv3·LGPLv3부터 추가되었습니다.
2차 저작물 공개 의무가 있는 GPL만 독점(폐쇄형) SW와의 결합이 불가능합니다.

라이선스 양립성 (Compatibility)

공개해야 하는 '범위'가 다르면, 결합 자체가 불가능해질 수 있습니다

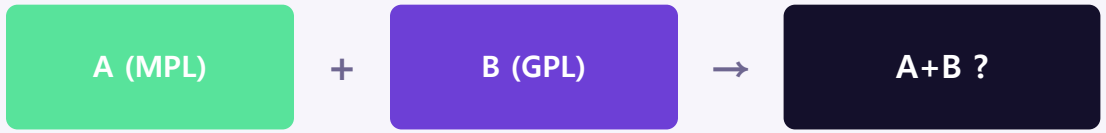
MPL — 파일 단위 공개

내가 고친 파일만 공개하면 충분. 같은 프로그램에 있는 다른 파일은 비공개(상용) 라이선스를 써도 무방

GPL — 전체 프로그램 공개

결합된 프로그램 전체를 GPL로 공개해야 함. '일부만 공개'는 허용되지 않음

예시 — MPL 코드(A) + GPL 코드(B)를 결합해 'A+B' 프로그램을 배포하려는 경우



GPL(B)의 요구를 지키려면 A도 GPL로 공개해야 하는데, **A는 애초에 MPL로만 공개하기로 허락된 코드라 GPL로 재배포할 권한이 없습니다. → 결국 'A+B'는 배포 자체가 불가능해집니다.**

어느 한쪽 요구를 만족하면 반드시 다른 쪽을 어기게 되는 구조 — 이게 '양립 불가능'입니다.

Apache License 2.0 ↔ GPLv3 — 양립 가능

GPLv3에서 호환성 조항이 추가되며 결합·배포가 가능해졌습니다.

MPL 1.1 ↔ GPLv3 — 양립 불가능

단, 2012년 나온 MPL 2.0은 GPL·LGPL과 결합해 재배포하는 것을 명시적으로 허용하는 조항을 추가해, 이 문제를 해결했습니다.

Permissive 라이선스(Apache, BSD, MIT 등)를 쓰는 경우, 일반적으로 양립성 문제를 크게 고려하지 않아도 되는 경우가 많습니다. 다만 최종 판단은 법적 전문가와 상담하는 것이 바람직합니다.

이중(Dual) 라이선스

하나의 코드에 두 가지 라이선스를 걸어, 사용자가 유리한 쪽을 고를 수 있게 합니다

'GPL or MIT' 형식

해당 소스코드를 GPL로 가져오거나, MIT로 가져올 수 있습니다.

상용 SW를 개발 중이라면 → 소스코드 공개 의무가 없는 MIT를 선택해 사용

커뮤니티에 계속 기여하고 싶다면
→ GPL을 선택해 파생물도 오픈소스로 유지

'GPL or Commercial' 형식

Commercial 라이선스를 구입하면, GPL의 소스코드 공개 등 의무사항 없이 상용 SW에 포함할 수 있습니다.

실제 사례 — MySQL

GPL 2.0으로 배포되지만, 상용 SW에 MySQL을 포함하려면 상용 라이선스를 구입하도록 하고 있습니다.

5 · 실무에 적용하기

라이선스 실무 관리 (Governance)

기업은 어떻게 수백 개의 오픈소스 라이선스를 관리할까

1 사용할 OSS 판단 → 2 라이선스 양립성 검토 → 3 사용 목록 작성 · 저장 → 4 소스 공개 · 사용여부 고지

외주업체가 개발한 코드에도 동일한 절차를 적용해야 합니다.

위반 시 벌어지는 일

FSF, SFLC(Software Freedom Law Center), KOSS Law Center 같은 단체가 라이선스 위반을 모니터링하고, 시정을 요구합니다. 기업이 거부하거나 시정이 미흡하면 소송으로 이어질 수 있습니다.

자주 묻는 질문

공개 SW 라이선스 가이드(NIPA)의 FAQ 중 실무에서 가장 자주 나오는 4가지

Q. 그냥 쓰기만 해도 의무가 생기나요?

배포하지 않는다면 대부분의 라이선스에서 의무가 발생하지 않습니다. 수정 후 배포할 때 비로소 라이선스에 따른 소스 공개 의무가 생깁니다.

Q. 상용 SW에 활용하려면 어떤 라이선스가 유리한가요?

소스공개 의무가 없는 BSD·MIT 같은 Permissive 라이선스가 유리합니다.

Q. 기업 도입 전 가장 먼저 확인할 것은?

2차 저작물의 소스 공개 의무가 있는 라이선스(GPL, LGPL, AGPL, EPL 등)인지부터 확인해야 합니다. 대체 불가하다면 공개를 감수하거나 상용 라이선스를 구매합니다.

Q. '소스 공개'는 정확히 무엇을 뜻하나요?

공개SW를 이용한 2차 저작물의 소스코드를 공개하는 것입니다. 공개 범위는 라이선스마다 다릅니다 — GPL은 링크된 코드까지, LGPL은 링크된 코드는 제외.

정리하며

라이선스는 자유를 막는 규칙이 아니라,
자유가 계속 이어지도록 지키는 약속입니다.

참고문헌

- 「공개 SW 라이선스 가이드」, 정보통신산업진흥원(NIPA)
- 「Open Source Software and GitHub」, 이민석, 국민대학교
- 「기업이 알아야 할 오픈소스 라이선스」, Black Duck Software Korea
- Black Duck, 「2026 Open Source Security and Risk Analysis(OSSRA)」 Report
- Open Source Initiative, opensource.org — 2025 License Usage Ranking